

TRABAJO PRÁCTICO: ANÁLISIS Y MEJORAS DE CÓDIGO (TP FINAL CONSOLIDADO)

UNIVERSIDAD TECNOLÓGICA NACIONAL (UTN) — EXTENSIÓN ÁULICA NECOCHEA

METODOLOGÍA DE SISTEMAS II

- **Proyecto:** Portal Web de la Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra" (Necochea)
 - **Docente:** Daniel Moreno
 - **Fecha de Entrega:** Junio 2026 · Necochea, Argentina
-

MIEMBROS DEL EQUIPO Y ROLES ASIGNADOS

Para la planificación, desarrollo y optimización del portal web, se definieron roles basados en las fortalezas individuales de los integrantes, asegurando una metodología de trabajo ágil y colaborativa:

- **Aramis Prieto (Scrum Master & Lead Backend Developer):** Responsable de facilitar las ceremonias ágiles, orquestar los modelos e interconexión de bases de datos híbridas, controlar la transaccionalidad concurrente mediante bloqueos de fila relacionales y programar la suite de pruebas de integración automatizadas.
 - **Kevin Nielsen (Security & Backend Developer):** Encargado de implementar la autenticación segura por JSON Web Tokens (JWT), configurar middlewares de control de tasa de solicitudes (Rate Limiting) por IP, estructurar validaciones mediante expresiones regulares y desarrollar el servicio de correos automatizados SMTP / REST API.
 - **Santiago Ialungo (UI/UX & Lead Frontend Developer):** Responsable del diseño visual adaptado al entorno de salud (paleta clínica, fondos de electrocardiograma ECG), maquetación responsiva con Tailwind CSS de las vistas públicas y el panel administrativo, e integración de Lenis scroll y Navbar Scroll-Spy.
 - **Thiago Masson (Full Stack Developer & Branding Specialist):** Encargado de la migración y estandarización del monorepo mediante entornos pnpm, integración de logotipos oficiales de la cooperadora, desarrollo del módulo documental de noticias e implementación de capas de sanitización contra ataques XSS con DOMPurify.
-

1. DESCRIPCIÓN GENERAL DEL ALCANCE DEL PROYECTO

El proyecto consiste en el diseño, desarrollo e implementación de un portal web interactivo, robusto y con arquitectura en la nube para la **Asociación Cooperadora del Hospital Municipal "Dr. Emilio Ferreyra"** de la ciudad de Necochea. El alcance abarca la digitalización de los procesos tradicionales de la institución, reemplazando las planillas físicas de papel por un sistema digital moderno, seguro y auditable.

Requerimientos generales planteados:

- **Gestión y Autogestión de Socios:** Registro en línea de usuarios, provistos de un panel privado ("Mi Panel") donde cada socio puede completar y actualizar sus datos obligatorios (DNI único, domicilio, teléfono) para ser incorporado formalmente al Libro Registro de Asociados digital tras la validación de la administración.
- **Módulo Transaccional de Cuotas Sociales:** Historial de pagos mensuales por parte de los socios, control automático de estados de membresía (activo/pendiente) y pasarela de cobro recurrente mediante suscripciones por débito automático.
- **Campañas de Recaudación Transparentes:** Visualización pública de las campañas de recaudación activas para la compra de insumos médicos o aparatología, provistas de barras de progreso financiero calculadas en tiempo real frente a metas de recaudación objetivas.
- **Declaración y Auditoría de Donaciones:** Checkout interactivo para registrar transferencias bancarias manuales mediante la carga del comprobante de pago, y un sistema alternativo de donación directa en línea a través de la pasarela de pagos.
- **Panel Administrativo:** Interfaz de control exclusiva para operadores y administradores que les permite validar transferencias manuales, dar de alta/baja/modificar campañas, y gestionar el estado del padrón de socios.
- **Módulo Informativo de Actualidad:** Portal dinámico de publicación de novedades y logros institucionales (obras concretadas, insumos adquiridos) para rendición de cuentas a la comunidad de Necochea y Quequén.

2. DESCRIPCIÓN DE LA ARQUITECTURA SELECCIONADA

Se ha optado por una **Arquitectura Cliente-Servidor Desacoplada** apoyada en una **Persistencia Híbrida** para responder de manera óptima a las necesidades transaccionales y multimedia del portal, garantizando robustez y rendimiento:

```

graph TD
    Cliente[Cliente: React SPA] -->|Peticiónes HTTPS + JWT| Backend[API Gateway: Node/Express]
    Backend -->|Persistencia Relacional ACID| SQL[(SQL: PostgreSQL / MySQL)]
    Backend -->|Esquema Documental Flexible| NoSQL[(NoSQL: MongoDB Atlas)]

    subgraph Base Relacional SQL (Sequelize)
        SQL -->|Usuarios e Identidad| users[usuarios]
        SQL -->|Libro Oficial de Socios| members[perfiles_socios]
        SQL -->|Finanzas y Metas| campaigns[campanas_eco]
        SQL -->|Trazabilidad Bancaria| payments[pagos_cuotas]
    end

    subgraph Base Documental NoSQL (Mongoose)
        NoSQL -->|Noticias e Hilos Multimedia| news[noticias_actualidad]
        NoSQL -->|Narrativa e Imágenes de Campaña| enrich[campanas_detalle]
    end

```

- **Lenguaje y Entorno:** Uso de **JavaScript (ES6+)** de extremo a extremo, utilizando **Node.js (Express)** en el backend y **React.js (Vite)** en el frontend. Esto unifica el stack tecnológico y optimiza la velocidad del desarrollo.
- **Frameworks y Librerías Base:** **Sequelize** como ORM para la interacción relacional y **Mongoose** como ODM para la base de datos documental.
- **Repositorio y Monorrepo:** Alojado en GitHub y orquestado mediante workspaces de **pnpm** para agilizar la consistencia de dependencias entre el cliente y el servidor en un único archivo de bloqueo consolidado (`pnpm-lock.yaml`).
- **Arquitectura de Datos Híbrida:**
 1. **Motor Relacional (SQL - PostgreSQL / MySQL):** Resguarda la información transaccional sensible bajo estrictas garantías **ACID**. Contiene las tablas de `usuarios` (credenciales hasheadas con `bcryptjs` y roles `admin` o `socio`), `perfiles_socios` (datos registrales) y `campanas_eco` (totales financieros). Emplea bloqueo exclusivo de filas (`SELECT ... FOR UPDATE`) en actualizaciones concurrentes de saldo para evitar condiciones de carrera.
 2. **Motor Documental (NoSQL - MongoDB Atlas):** Almacena estructuras dinámicas de formato libre y alta carga multimedia. Contiene las colecciones de `noticias_actualidad` y `campanas_detalle` (historias, testimonios y galerías de fotos) vinculadas lógicamente mediante identificadores de referencia.
 3. **Data Mashup (Promise.all):** Para reducir latencias de red, el backend ejecuta consultas en paralelo en ambos motores mediante `Promise.all` , unificando la respuesta financiera (SQL) y descriptiva (NoSQL) en un solo objeto JSON plano enviado al cliente en una única petición.

3. ESTADO ACTUAL DEL CÓDIGO Y PROPUESTAS DE MEJORA

A continuación, se detalla el estado actual del código y se detectan bloques de oportunidad para incorporar buenas prácticas mediante patrones de diseño creacionales, estructurales y de comportamiento.

3.1. Patrón Singleton (Creacional)

- **Módulo / paquete:** `backend/config/db.js` y `backend/config/mongo.js` (Conexiones a las bases de datos).
 - **Solución a implementar:** Implementar el patrón **Singleton** para la clase o módulo que expone la conexión de base de datos a Sequelize y Mongoose. Asegura que toda la aplicación web comparta una única instancia de conexión y que las llamadas subsiguientes recuperen la misma referencia en lugar de reabrir canales de sockets.
 - **Ventajas de la implementación:**
 - **Optimización de recursos:** Ahorro sustancial de memoria y prevención de la sobrecarga del pool de conexiones al evitar múltiples instancias innecesarias.
 - **Consistencia:** Facilita la transaccionalidad al asegurar que todos los modelos operen sobre el mismo hilo de conexión.
 - **Posibles desventajas:**
 - Dificulta los tests unitarios de integración al introducir estado global en la aplicación, obligando a realizar mocks del estado de conexión o a limpiar las instancias al finalizar cada suite.
-

3.2. Patrón Factory Method (Creacional)

- **Módulo / paquete:** `backend/services/authService.js` (Lógica de creación y registro de cuentas de usuario).
- **Solución a implementar:** Actualmente, el registro de usuarios evalúa condicionales directos para instanciar el perfil según el tipo de rol (`socio` o `admin`). Se propone implementar el patrón **Factory Method** a través de una clase fábrica (`UserFactory`). Esta clase, al recibir el rol y los datos provistos en el formulario, devolverá la instancia correcta del modelo de usuario con sus respectivos campos obligatorios y permisos por defecto.
- **Ventajas de la implementación:**
 - **Principio de Responsabilidad Única (SRP):** Centraliza y encapsula la lógica de instanciación de usuarios, aislando el controlador de autenticación de los detalles de construcción de modelos.
 - **Principio Abierto/Cerrado (OCP):** Permite añadir nuevos tipos de perfiles en el futuro (ej. operador administrativo intermedio, auditor externo) simplemente heredando de la clase base y extendiendo la fábrica sin alterar el backend core.
- **Posibles desventajas:**

- Podría generar una sobreingeniería si la plataforma mantiene un esquema estático con únicamente dos roles fijos (`socio` y `admin`) durante su ciclo de vida.

3.3. Patrón State + Asincronía (Comportamiento)

- **Módulo / paquete:** `backend/controllers/donacionController.js` (Flujo de auditoría y validación de transferencias bancarias).
- **Problema detectado:** Actualmente, cuando un donante declara una transferencia, el sistema la almacena en estado "pendiente". El progreso financiero de la campaña y el mail de agradecimiento quedan suspendidos hasta que un administrador ingresa manualmente al panel de control, verifica la acreditación del dinero en el homebanking y presiona el botón "Aprobar". Esto ralentiza la actualización en tiempo real de las métricas públicas de recaudación y congela la interacción con el usuario.
- **Solución a implementar:** Implementar el patrón **State (Estado)** combinado con una arquitectura orientada a eventos mediante un **Worker asíncrono**:
 1. La donación encapsula su comportamiento de transición de estados a través de una interfaz común, delegándolo a clases concretas (`EstadoPendiente` , `EstadoAprobado` , `EstadoRechazado`).
 2. Diseñar un servicio de conciliación automática en segundo plano (Worker / Cron Job) que consuma un webhook simulado de la entidad bancaria del hospital. Al detectar el ingreso de fondos, el sistema transiciona la donación asíncronamente a `EstadoAprobado` , actualiza la barra de progreso y dispara el correo de confirmación de forma desatendida.
- **Ventajas de la implementación:**
 - **Desacoplamiento:** La lógica compleja asociada al cambio de estado se aísla en clases específicas, evitando condicionales anidados en el controlador.
 - **Mejora operativa y de UX:** Las barras de progreso se actualizan inmediatamente al acreditarse los fondos y el donante recibe feedback rápido sin depender de la disponibilidad horaria del personal.
- **Posibles desventajas:**
 - Incrementa la complejidad técnica del backend al requerir el manejo de tareas programadas en segundo plano y colas de reintentos en caso de fallos de red.

3.4. Patrón Factory + Strategy en Webhooks (Comportamiento/Estructural)

- **Módulo / paquete:** `backend/controllers/socioSubscriptionController.js` (Webhook unificado de Mercado Pago).
- **Problema detectado:** El controlador encargado de recibir los eventos de Mercado Pago (`webhookMercadoPago`) realiza múltiples tareas: valida firmas criptográficas de seguridad, mitiga ataques de repetición y procesa tanto notificaciones de socios (`preapproval`) como pagos de cuotas y donaciones de campañas (`payment`). Esto genera un alto acoplamiento inter-módulo (el controlador de socios maneja e importa modelos de campañas).

- **Solución a implementar:** Extraer la gestión de webhooks a un controlador independiente y desacoplar la lógica de procesamiento aplicando **Factory** y **Strategy**:
 1. Definir una interfaz común `WebhookHandler` con el método `process(payload)`.
 2. Implementar estrategias concretas: `PreapprovalWebhookHandler` y `PaymentWebhookHandler`.
 3. Crear una fábrica `WebhookHandlerFactory` que instancie la estrategia correcta según el tipo de evento recibido en la cabecera de Mercado Pago.
- **Ventajas de la implementación:**
 - **Modularidad:** El controlador de webhooks solo actúa como enrutador y validador de cabeceras de seguridad, delegando la lógica de negocio.
 - **Escalabilidad:** Agregar nuevos eventos de Mercado Pago (ej. cancelaciones, disputas) consiste únicamente en crear una nueva estrategia e inscribirla en la fábrica.
- **Posibles desventajas:**
 - Aumenta el número de clases y archivos pequeños en la estructura de controladores del backend.

3.5. Capa de Servicios / Service Layer (Estructural)

- **Módulo / paquete:** `backend/controllers/donacionController.js` (Gestión transaccional en `aprobarTransferencia`).
- **Problema detectado:** El método `aprobarTransferencia` realiza tareas a múltiples niveles: analiza la cabecera HTTP, gestiona transacciones Sequelize, aplica bloqueos exclusivos de fila en base de datos (`lock`), ejecuta validaciones de lógica de negocio, purga la caché en memoria y envía correos electrónicos. Esto sobrecarga de responsabilidades al controlador.
- **Solución a implementar:** Extraer la lógica del negocio del controlador a una capa intermedia de servicios (`donacionService.js`). El controlador de Express solo se encargará de parsear la petición HTTP, llamar a la función de servicio y formatear la respuesta JSON o capturar excepciones del dominio.
- **Ventajas de la implementación:**
 - **Reutilización:** Permite invocar la misma lógica de aprobación de transferencias desde scripts de consola, test unitarios o webhooks sin requerir simular peticiones HTTP (`req`, `res`).
 - **Testabilidad aislada:** Simplifica el testeo unitario de las reglas de negocio usando mocks.
- **Posibles desventajas:**
 - Exige refactorizar las llamadas de base de datos a lo largo de todos los controladores para mantener la homogeneidad arquitectónica.

3.6. Patrón Observer / Event-Driven (Comportamiento)

- **Módulo / paquete:** `backend/controllers/donacionController.js` y `backend/controllers/socioController.js` (Efectos secundarios de aprobaciones).

- **Problema detectado:** Cuando un administrador aprueba una donación o activa a un socio, el controlador dispara secuencialmente el envío de correos electrónicos transaccionales y la invalidación de la caché en memoria. Si el servidor de correos (Resend) sufre una demora o falla, la respuesta al usuario se bloquea o retorna un error 500, a pesar de que el registro en la base de datos se completó correctamente.
- **Solución a implementar:** Implementar el patrón **Observer** utilizando la clase nativa `EventEmitter` de Node.js. Cuando ocurre un cambio de estado en el dominio, el servicio emite un evento (ej. `donation.approved` o `socio.activated`). Módulos observadores (listeners) independientes reaccionarán a dicho evento de forma asíncrona y no bloqueante.
- **Ventajas de la implementación:**
 - **Desacoplamiento total:** La lógica core del negocio no sabe qué sistemas externos reaccionan a sus acciones.
 - **Resiliencia (Rendimiento non-blocking):** La petición HTTP del usuario finaliza instantáneamente sin esperar el envío físico de correos o la actualización de caché de APIs externas.
- **Posibles desventajas:**
 - El flujo de ejecución secuencial se transforma en un paradigma reactivo orientado a eventos, lo que puede elevar levemente la curva de depuración del flujo para programadores noveles.